

AA-3

Retrieval Augmented Generation System for Internal Knowledge Management

Applied Artificial Intelligence Laboratory Ltd.

Group 1

Frederick Khasanto 122040014

Matthew Pranata 122040035

Stefan Susanto 122040041





BACKGROUND



- Internal knowledge is stored in Notion PDF exports
- Users need fast and accurate answers from manuals
- Manually reading many documents is slow
- Traditional keyword search may miss relevant context
- Standalone LLMs may hallucinate without document grounding





PROBLEM STATEMENT

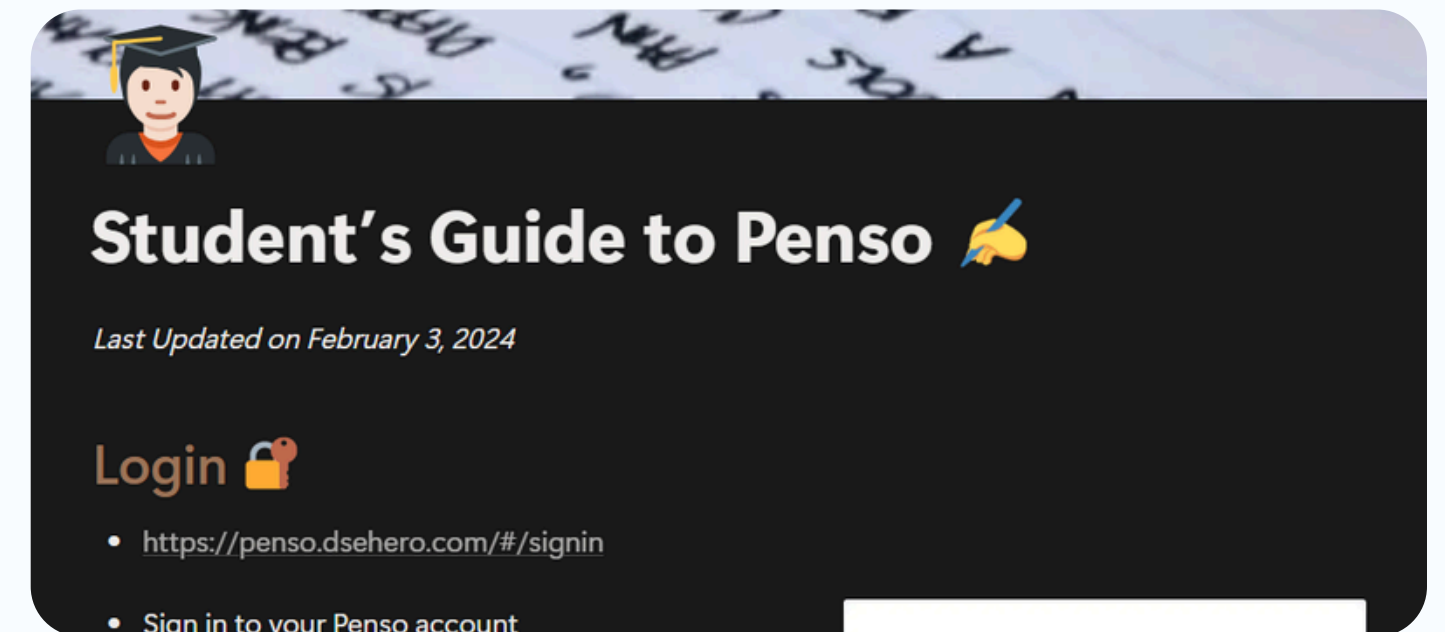
- Internal documents are unstructured and hard to search
- Users often need to open multiple files for one answer
- Keyword-based search cannot fully understand user intent
- Normal chatbots may generate unsupported answers
- High latency reduces real-time usability

We need a system that can search internal documents semantically and answer questions based on trusted source materials.

About

Progress

Future





SOLUTION

We built a
**Retrieval-Augmented
Generation (RAG) Chatbot**
powered by internal
Notion PDF documents



The system retrieves relevant document chunks before generating answers



Responses are grounded in internal Sayo documentation



Source links are returned using stored metadata



The final system supports English, Chinese, and mixed-language queries



WHY DO WE NEED RAG?

Welcome to

```
EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II     ZZ     AA  AA
EEEEEE LL      II     ZZZ    AAAAAA
EE      LL      II     ZZ     AA  AA
EEEEEE LLLLLL  IIII  ZZZZZZ  AA  AA
```

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?

YOU: Men are all alike.

- Relies on pre-trained knowledge
- May generate incorrect or fabricated answers
- Cannot access internal company data
- No source transparency (cannot verify where answers come from)

RAG

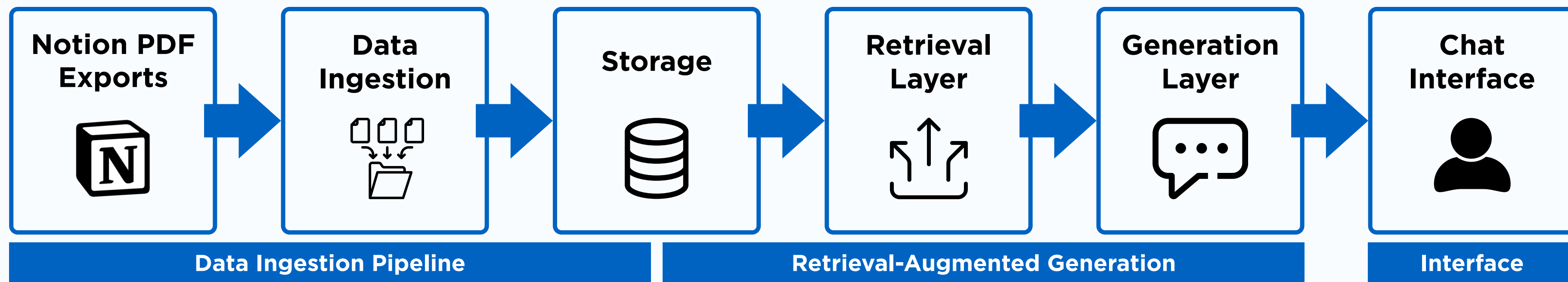
Chatbot



- Retrieves relevant internal documents
- Generates answers from real data
- Accurate and source-grounded responses
- RAG bridges the gap between LLM intelligence and reliable internal knowledge.

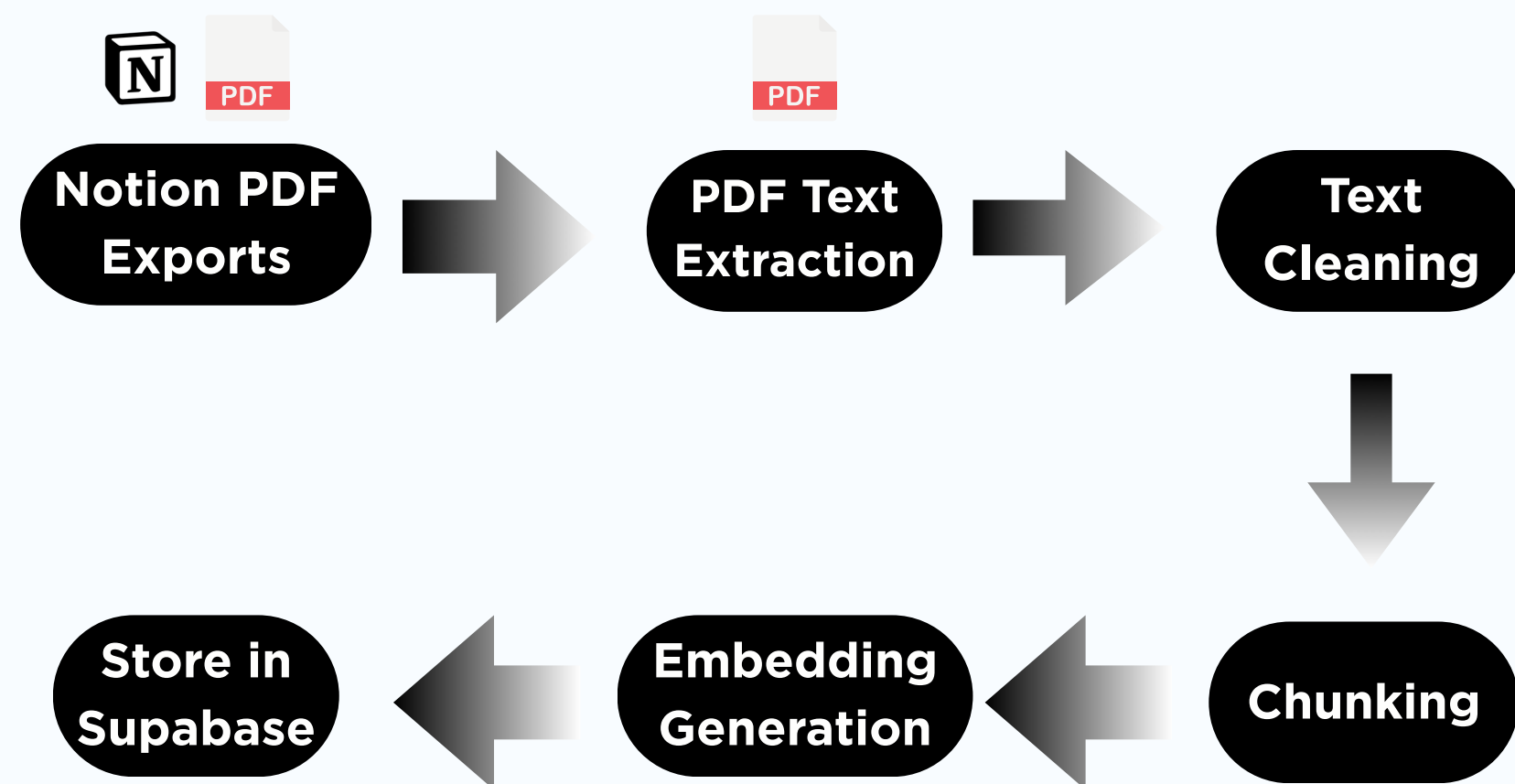


SYSTEM ARCHITECTURE





DATA INGESTION PIPELINE



TOOL	PROBLEM
EXTRACTION	Read Text From PDF Pages
CLEANING	Remove repeated headers, footers, and noisy lines
CHUNKING	Split text into 500-token segments
EMBEDDING	Convert chunks into vectors using pplx-embed-v1-4b
STORAGE	Save chunks and metadata in Supabase



TEXT EXTRACTION AND CLEANING

- PDFs processed using PyMuPDF
- Text extracted page by page
- OCR fallback used when a page has no selectable text
- Cleaning removes:
 - Repeated headers/footers
 - Page numbers
 - Empty lines
 - Extra whitespace





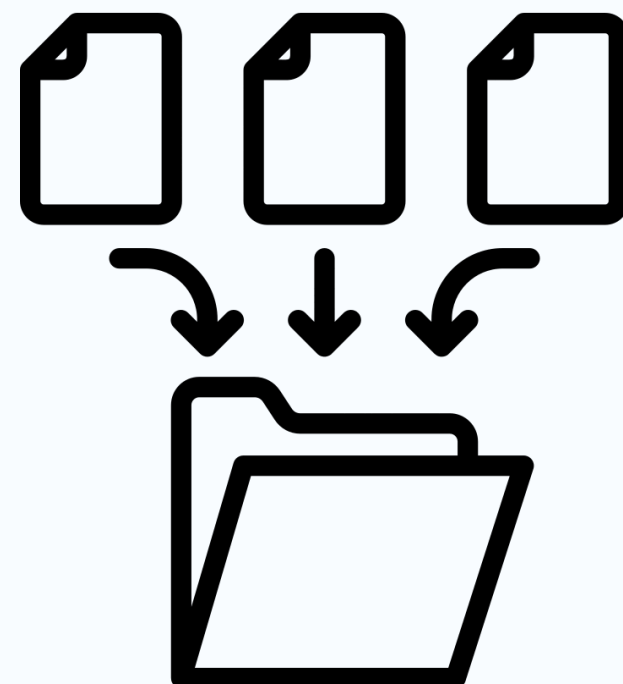
CHUNKING

- **Chunk size: 500 tokens**
- **Overlap: 50 tokens**
- **Chunks are created to preserve context**
- **Sentence-aware chunking is used to reduce unnatural splits**
- **Each chunk stores metadata:**
 - **chunk_index**
 - **page_number**
 - **chunk_total**
 - **document_title**
 - **manual_type**
- **Too large → retrieval becomes noisy**
- **Too small → context is lost**





INGESTION OUTPUT



- Cleaned text chunks extracted from Notion PDF manuals
- Each chunk has 500 tokens with 50-token overlap
- Each chunk is converted into a vector embedding (1536 dimensions)
- Metadata is attached to each chunk:
 - Document title
 - Source URL
 - Manual type (teacher / student)
 - Page number
 - Chunk index
- The processed chunks, embeddings, and metadata are then passed to the storage layer for indexing and retrieval



STORAGE LAYER

- Vector DB: Supabase (pgvector)
- Why chosen: scalability, integration ease
- Database Schema

documents	:
id	uuid
content	text
embedding	vector
document_title	text
source_url	text
manual_type	text
created_at	timestamp
chunk_index	int4
page_number	int4
chunk_total	int4

Knowledge Base (Supabase)



- Document chunks
- Vector search
- Metadata



supabase

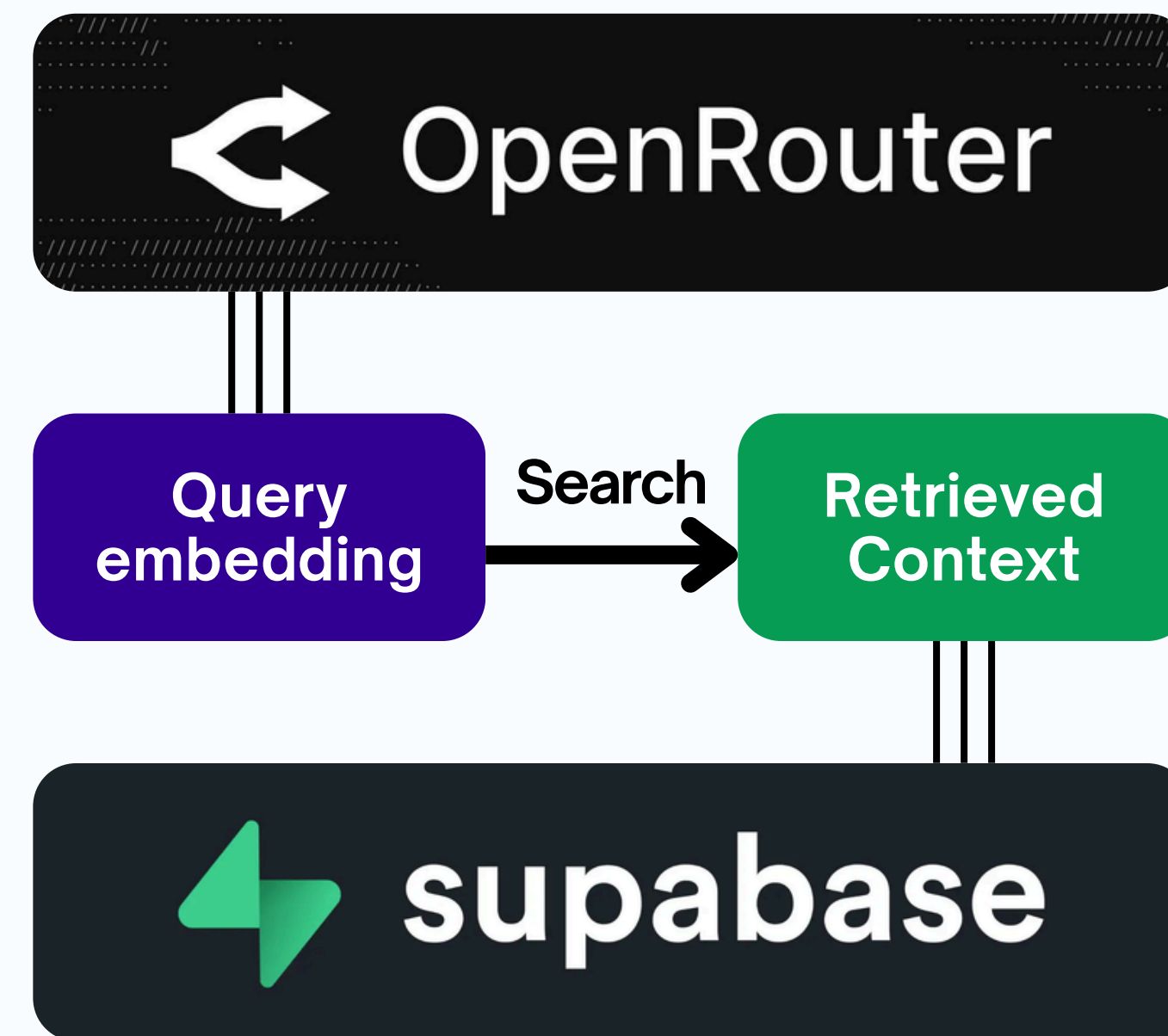


PostgreSQL + pgvector



RETRIEVAL LAYER

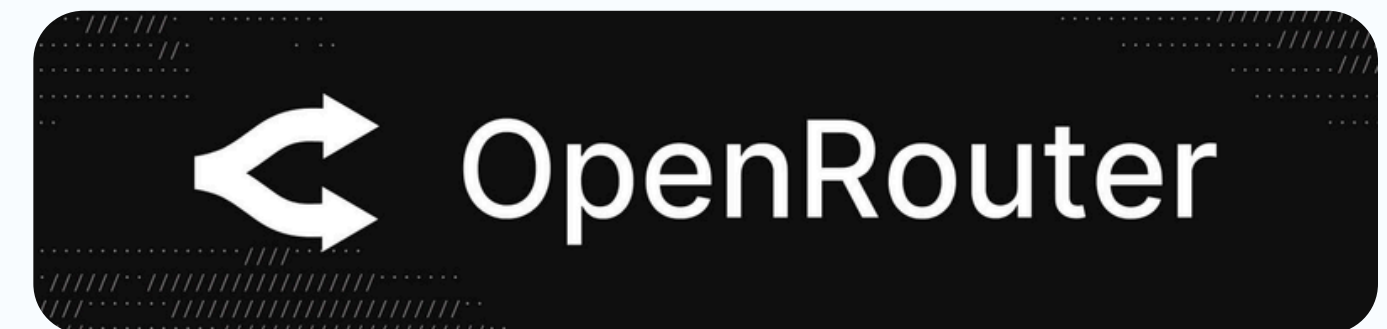
- Input: User query
- Processing:
 - Query $\rightarrow$ embedding vector
 - Perform similarity search in vector DB
 - Optional keyword-based filtering
- Retrieval Strategy:
 - Hybrid Search:
 - Semantic (vector similarity)
 - Keyword matching
- Output:
 - Top-K relevant document chunks





GENERATION LAYER

- Process:
 - Combine **System Query + Retrieved context + User query**
 - Construct *prompt template*
- Generation:
 - LLM produces grounded response
 - Context-aware and fact-based
- Output:
 - RAG Answer with source URL



[System Prompt]
[Retrieved Context]
[User Query]

Prompt Template



GENERATION LAYER

You are a **helpful** AI support assistant for **sayo.ai**, an educational platform for teachers and students. Your role is to provide accurate, comprehensive answers based **ONLY** on the provided tutorial documents.

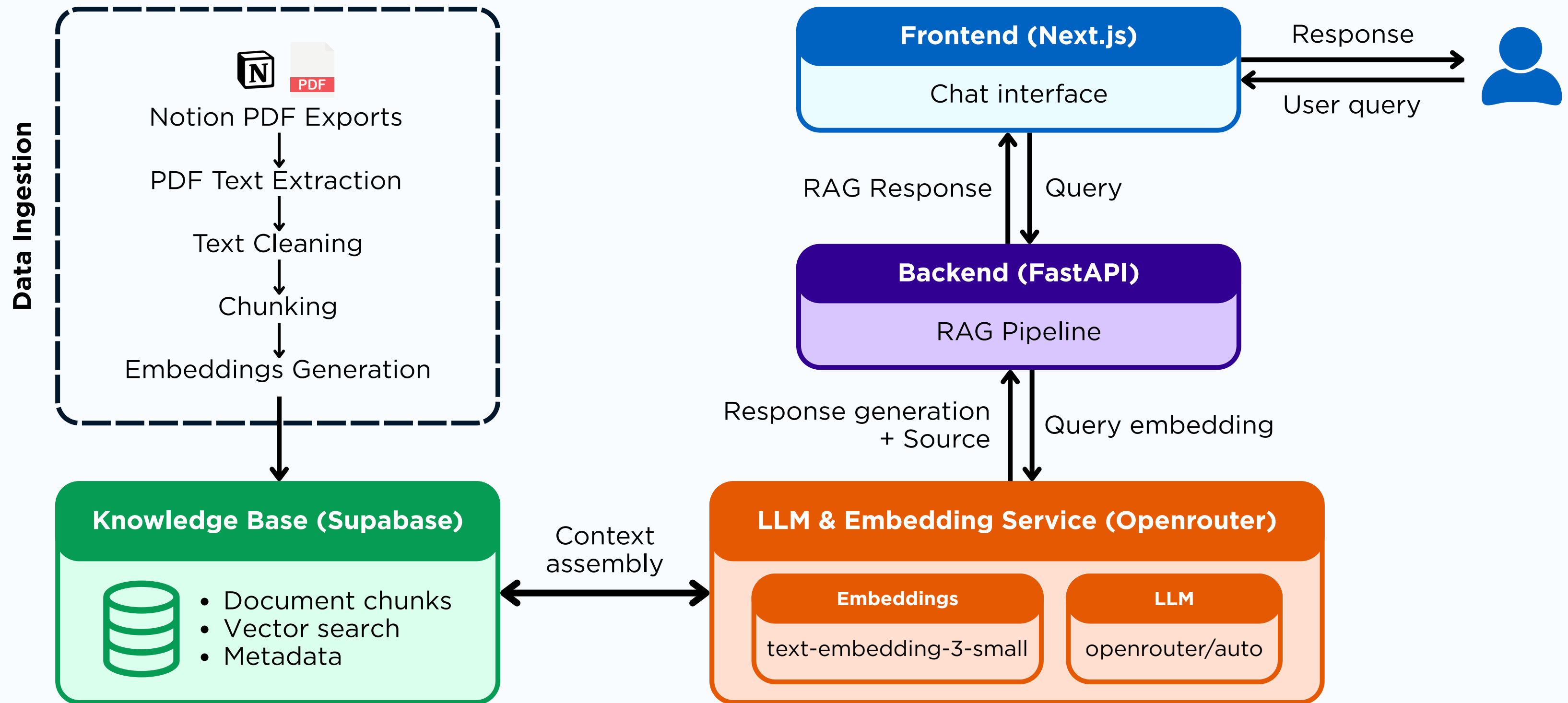
Guidelines:

1. **RESPONSE ACCURACY**: Always base your answers on the provided context documents. If the documents contain a step-by-step guide, provide the complete sequence precisely.
2. **SOURCE CITATION**: Do NOT include document names or links within the body of your response or at the end of each sentence. Instead, provide all relevant sources at the very end of your response in a single, well-formatted sentence.
3. **CITATION FORMAT**: Use the following format for the citation sentence: "For more details, please refer to: [Document Title 1] (URL1), [Document Title 2](URL2)."
4. **RELEVANCE**: Only cite documents that you actually used to answer the question. If a document is in the context but unrelated to the specific question, ignore it.
5. **HONESTY**: If the answer is NOT in the provided materials, say: "I don't have information about this in the Sayo documentation. Please contact Sayo support or check the latest tutorials."
6. **FOCUS**: Do not use outside knowledge. Do not make assumptions. DO NOT MAKE artificial websites or links.
7. **FORMATTING**: Use clear bullet points or numbered lists for instructions. Keep the tone professional yet helpful.

System Prompt

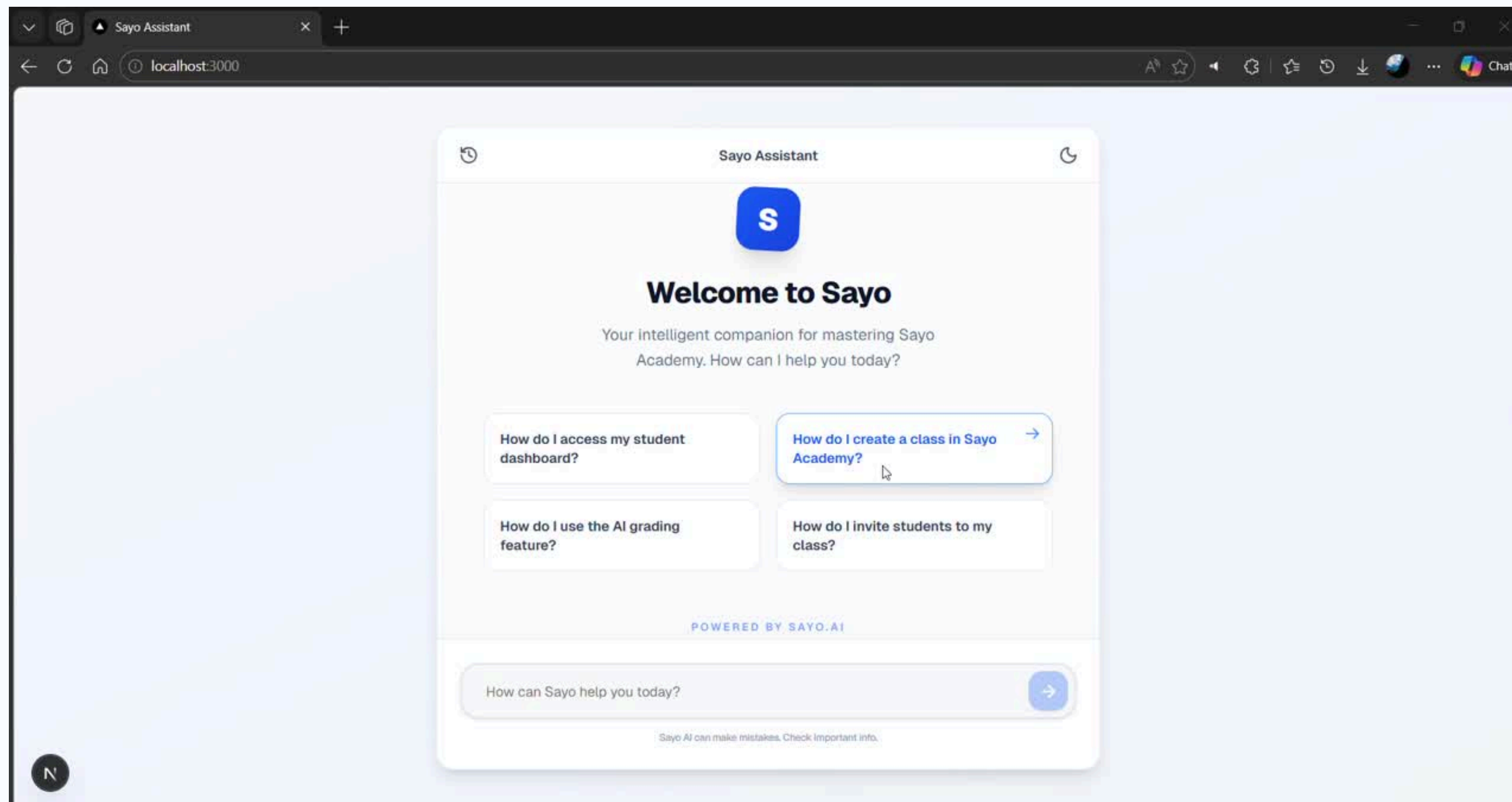


DATA FLOW PIPELINE





DEMO VIDEO

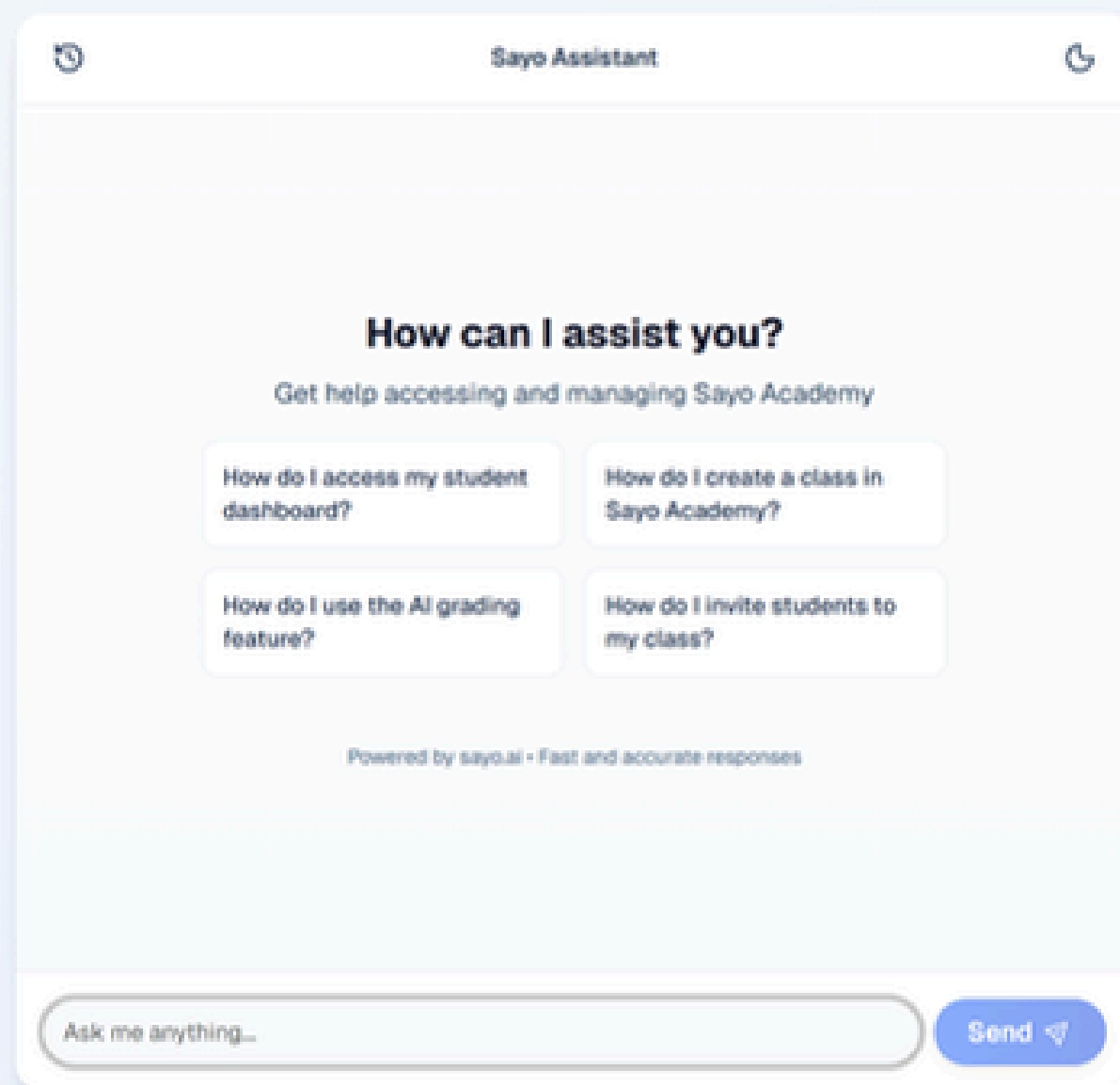




UI INTERFACE

Next.js 14 premium chat UI with:

- **Markdown & Code Rendering:** Full support for bullet points, headers, tables, and clickable links via `react-markdown`.
- **Syntax Highlighting:** Beautiful code blocks with language-specific highlighting and a copy-to-clipboard button.
- **ChatGPT-like Aesthetics:** Modern card-based welcome screen, premium input area with backdrop-blur, and improved message layouts.
- **Interactive History Sidebar:** Slide-in panel to manage and switch between past conversations.
- **Dark Mode:** Complete support for both light and dark themes with a quick toggle.

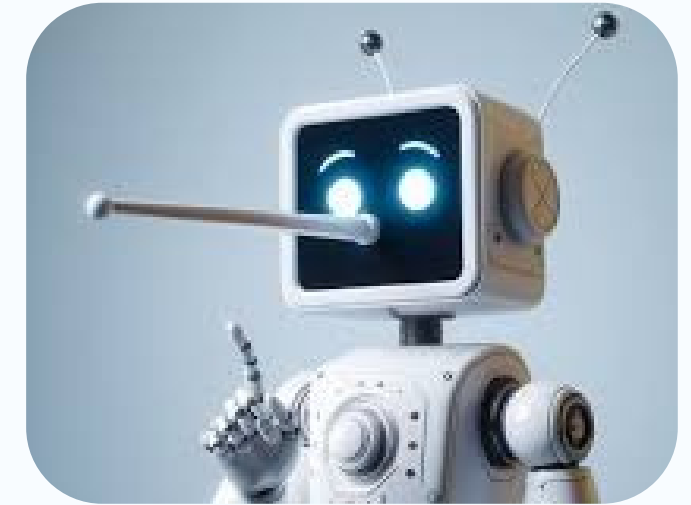




CURRENT CHALLENGES

HALLUCINATIONS

Despite operating within a RAG framework, the model continues to generate responses that are factually incorrect or unsupported by our documents. This might be caused by poor upstream retrieval quality. Although prompt engineering has been applied as a mitigation measure, the issues persists in some cases.



NON-ROBUST CONTEXT SEARCH ALGORITHM

Within our RAG pipeline, the retrieval component fails to consistently search the most semantically relevant document given a query. This might be caused by our use of cosine similarity retrieval, which lacks the semantic depth to more accurately capture the query intent.





NEXT S



AA-3

Retrieval Augmented Generation System for Internal Knowledge Management

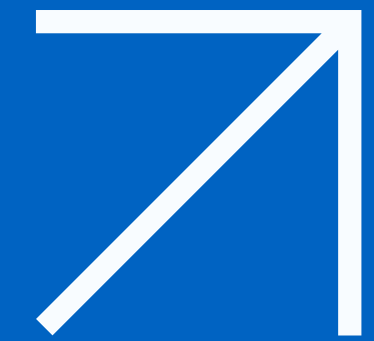
Applied Artificial Intelligence Laboratory Ltd.

Group 1

Frederick Khasanto 122040014

Matthew Pranata 122040035

Stefan Susanto 122040041



Overcome Existing

We will work on current issues: retrieval accuracy, inconsistency, and limited UI/UX.

improve accuracy, consistency, and system reliability.

support continuous updates and reliable performance.

Deployment

minimize our system for larger deployment. We plan to deploy our RAG on a scalable, cloud infrastructure. We plan to deploy our RAG on a scalable, cloud infrastructure. We plan to deploy our RAG on a scalable, cloud infrastructure.



CONCLUSION

- We developed a RAG-based chatbot for internal knowledge management
- Enables fast, accurate, and document-grounded answers
- Reduced latency from ~30s to 5–7s
- Improved answer reliability and source correctness
- Supports multilingual queries (English, Chinese, mixed)

This demonstrates the real-world potential of RAG systems in enterprise applications.



THANK YOU

*AAIL-3 Retrieval Augmented Generation
for Internal Knowledge Management*